AFDPS for Navier-Stokes -- Setup & Run Guide

This guide takes you from a fresh GPU box to a first benchmark number and a full hyperparameter sweep for the afdps-ns project (AFDPS applied to the InverseBench 2D Navier-Stokes inverse problem). The numerical core is already built and verified on CPU (15/15 checks); the remaining work is GPU setup, asset download, and running Track B.

GitHub: https://github.com/erichou1/AFDPS (public -- clone it directly on the GPU box; no copying from a laptop). NOTE: on GitHub the repo is named AFDPS and the files are at the ROOT -- there is no 'afdps-ns' subfolder (that is only the local clone-directory name).

Goal: beat the InverseBench baselines on relative-L2 -- EnKG ~ 0.12, DPG ~ 0.32 (x2 subsampling, sigma=0).

Where things stand (what is already done)

- Repo built: InverseBench fork + vendored AFDPS sampler + the new adjoint gradient engine.
- Numerical core VERIFIED on CPU (float64): discrete-adjoint gradient matches finite differences to ~1e-10; Hutchinson Laplacian matches the brute-force trace; adjoint/FFT primitives exact. Run: python verification/checks.py -> 15/15.
- Track A (synthetic, analytic Gaussian prior) recovers the field end-to-end.
- Track B (real benchmark) code path is smoke-tested; it needs the checkpoint + data + GPU.

So the next steps are purely: get it on the GPU box, download assets, verify, then run.

Prerequisites

- A Linux GPU box (the GB200s). NVIDIA driver + CUDA runtime that matches your torch build.
- Python 3.9+ (3.10/3.11 recommended).
- ~5-10 GB disk for the checkpoint + datasets. Internet access to GitHub + Caltech data.
- (Optional) a Weights & Biases account for the sweep (wandb login).

Step 1 -- Get the repo onto the GPU box (git clone)

The project is on GitHub (public), so you do NOT copy anything from your laptop. On the GPU box, just clone it:

```
git clone https://github.com/erichou1/AFDPS.git
cd AFDPS
```

This gives you a folder 'AFDPS' containing the whole project (main.py, algo/, inverse_problems/, configs/, verification/, SETUP_GUIDE.pdf, ...). If the repo is later made private, authenticate first (gh auth login, or an SSH key / personal access token on the box).

Large assets are intentionally NOT in git (.gitignore excludes checkpoints/ and data/) -- you download those in Step 3. To pull later updates: git pull.

Step 2 -- Python environment

Two options. Use whichever your cluster prefers. Option A (uv) matches InverseBench.

Option A: uv (recommended)

```
curl -LsSf https://astral.sh/uv/install.sh | sh   # if uv is not installed
cd AFDPS                                          # the cloned repo
uv sync          # installs the NS-focused deps from pyproject.toml
source .venv/bin/activate
```

Option B: pip / conda

```
python -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
# install the CUDA build of torch that matches your driver, then the rest:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
pip install hydra-core omegaconf lmdb numpy tqdm piq pyyaml
# wandb is OPTIONAL (only for the Step 8 sweep): pip install wandb
```

Sanity check the GPU is visible:

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available(),
torch.cuda.get_device_name(0))"
```

NOTE: the trimmed pyproject.toml omits the FWI/MRI/black-hole solvers (devito, ehtim, fastmri, sigpy). That is intentional -- they are not needed for Navier-Stokes. If you ever run those other InverseBench problems, restore them from env.yaml / README_InverseBench.md.

Step 3 -- Download the assets (checkpoint + data)

One command downloads everything -- the ~102 MB diffusion prior AND the ~7 MB NS datasets -- and unzips the data to the exact paths the configs expect (needs curl + unzip):

```
bash scripts/download_assets.sh          # or: ... data   |   ... ckpt
```

Result: checkpoints/ns-5m.pt and ../data/navier-stokes-{test,val}/Re200.0-t5.0 (an LMDB dir with data.mdb + lock.mdb; 'data' is a SIBLING of the repo folder).

About the data access (CaltechDATA)

The NS data lives in a CaltechDATA record whose METADATA is public but whose FILES are RESTRICTED, so the bare record URL will not let you download. Access needs a share token -- the script uses the public one already in README_InverseBench.md. If the data step ever returns 401/403, the token expired: open the tokenized link in README_InverseBench.md (or get a fresh share link from the record page) and replace CALTECH_TOKEN in scripts/download_assets.sh.

The record has 4 versions. The NS test/val files are byte-identical across all of them, so the version does not matter for the benchmark; the downloader uses the LATEST (zg89b-mpv16), which also contains navier-stokes-train.zip (~1.24 GB) -- only needed if you retrain the prior (bash scripts/download_assets.sh train).

Manual equivalent (if you prefer)

```
# checkpoint:
curl -L -o checkpoints/ns-5m.pt \
  https://github.com/devzhk/InverseBench/releases/download/diffusion-prior/ns-5m.pt
# data (TOK = the token from README_InverseBench.md):
B=https://data.caltech.edu/api/records/jfdr4-6ws87/files
curl -L -o ns-test.zip "$B/navier-stokes-test.zip/content?token=$TOK"
curl -L -o ns-val.zip  "$B/navier-stokes-val.zip/content?token=$TOK"
mkdir -p ../data && unzip -o ns-test.zip -d ../data && unzip -o ns-val.zip -d ../data
```

Step 4 -- Verify the install

Run all three checks. The first two need no GPU and no checkpoint; the third needs neither the real checkpoint nor a GPU (it uses a mock prior to test the code path).

```
python verification/checks.py          # expect: 15/15 checks passed.
python verification/run_track_a.py --res 32 --steps 150 --particles 8
python verification/smoke_track_b.py   # expect: PASS (Track B code path executes)
```

- checks.py: the numerical verification ladder (gradient vs finite-diff, Hutchinson vs brute-force trace, adjoint/FFT primitives, GRF prior).

- run_track_a.py: end-to-end AFDPS on a synthetic Gaussian-prior problem; prints rel-L2 (should be well below the prior-mean baseline of 1.0).
- smoke_track_b.py: composes the real Hydra configs and runs one inference with a mock net.

Step 5 -- Validate the forward solver at full 128 resolution (open item)

The dataset was generated with ADAPTIVE time-stepping, but the adjoint needs a FIXED dt (adaptive=False, delta_t=0.002). Confirm delta_t=0.002 is sub-CFL at 128x128 BEFORE the full run -- if the forward diverges (NaN), lower delta_t (e.g. 0.001).

```
python -c "import torch,sys; sys.path.insert(0,'.');\
from inverse_problems.navier_stokes_afdps import AFDPSNavierStokes2d;\
op=AFDPSNavierStokes2d(resolution=128, forward_time=1.0, Re=200.0,\
  downsample_factor=2, delta_t=0.002, adaptive=False, unnorm_scale=10.0,\
  device='cuda');\
x=0.5*torch.randn(2,1,128,128,device='cuda');\
y=op.forward(x); print('finite:', torch.isfinite(y).all().item(), 'shape:', tuple(y.shape))"
```

Expect finite: True. If False, set problem.model.delta_t=0.001 everywhere below.

Step 6 -- First Track B run (one setting)

Run AFDPS on the real benchmark at x2 subsampling, sigma=0. This prints relative-L2 per test case and an aggregate at the end.

```
python main.py problem=navier-stokes-afdps algorithm=afdps pretrain=navier-stokes \
    num_samples=1 wandb=false
```

Results are saved under exps/inference/navier-stokes-afdps-ds2/AFDPS/. If you see NaN or rel-L2 >> 1, it is almost always the guidance strength -- go to Step 7.

Step 7 -- Tune the guidance (the single most important knob)

guidance_gamma controls the annealed data-guidance strength and MUST be scaled to the data. Too small -> unstable / NaN; too large -> ignores the data (rel-L2 ~ 1). Sweep it by hand first to find the stable, useful range, e.g.:

```
for g in 1 3 10 30; do \
  python main.py problem=navier-stokes-afdps algorithm=afdps pretrain=navier-stokes \
    algorithm.method.guidance_gamma=$g num_samples=1 wandb=false \
    exp_name=gamma_$g ; done
```

Other useful knobs (all overridable on the command line):

- algorithm.method.num_steps (e.g. 100/200/400) -- more steps = more stable, slower.
- algorithm.method.num_particles (8/16/32/64) -- ensemble size; batched on GPU.
- algorithm.method.sigma_max (40/80) -- top of the noise schedule.
- problem.model.hutchinson_M (1/2/4) -- # Laplacian probes; raise for less noisy weights.
- problem.model.hutchinson_scheme=forward -- halves the Laplacian solves (faster).
- problem.model.grad_chunk -- raise to fill the GPU (memory permitting).

Step 8 -- Hyperparameter sweep (OPTIONAL -- needs wandb)

This is the ONLY step that needs Weights & Biases (a free account + 'pip install wandb'). It is optional: everything else runs with wandb=false (the default) and no account. If you skip wandb, just tune by hand as in Step 7 (a shell loop over configs) -- results print to stdout and save under exps/.

With wandb: run the bayes sweep over the validation set.

```
wandb login                                  # once
```

```
wandb sweep configs/sweep/navier-stokes/afdps.yaml
# copy the printed sweep ID, then launch one or more agents (one per GPU):
wandb agent <ENTITY/PROJECT/SWEEP_ID>
```

The sweep optimizes 'relative l2' (the metric main.py actually logs on the AFDPS path) over num_particles, num_steps, guidance_gamma, sigma_max, hutchinson_M, hutchinson_scheme.

Step 9 -- Full benchmark grid + compare to baselines

Run the best config across the 3x3 grid: subsampling in {2,4,8} x noise in {0,1,2}. Each is a separate run via problem.model overrides. Example (x4, sigma=1):

```
python main.py problem=navier-stokes-afdps algorithm=afdps pretrain=navier-stokes \
    problem.model.downsample_factor=4 problem.model.sigma_noise=1.0 \
    algorithm.method.guidance_gamma=<tuned> num_samples=1 wandb=false \
    exp_name=ds4_sig1
```

Collect the aggregate relative-L2 for each cell and tabulate against the published baselines (EnKG, DPG, EKI, DPS-GSG). You can also run those baselines in this same repo for an apples-to-apples comparison, e.g.:

```
python main.py problem=navier-stokes algorithm=enkg pretrain=navier-stokes wandb=false
python main.py problem=navier-stokes algorithm=dpg  pretrain=navier-stokes wandb=false
```

(Baselines use problem=navier-stokes, the black-box forward op; AFDPS uses problem=navier-stokes-afdps, the adjoint-enabled op.)

Optional / later

- Continuous-adjoint cross-check: set problem.model.adjoint_mode=continuous to use the hand-coded backward PDE solver instead of autograd (agrees to O(dt); a good sanity check and a pedagogical artifact for the paper).
- Ablations: diffusion prior vs analytic GRF prior; hutchinson_M and particle-count scaling; central vs forward Hutchinson; guidance schedules.
- Extend to the inverse-scattering problem (InverseBench Appendix B.4) reusing the adapter pattern, if you want a second benchmark for the paper.

Troubleshooting

- NaN / rel-L2 >> 1: lower guidance_gamma (most common), or raise num_steps, or lower delta_t. The sampler now quarantines diverged particles, but a bad config still scores poorly.
- Forward diverges at 128^2: delta_t too large for CFL -> set problem.model.delta_t=0.001.
- CUDA out of memory: lower problem.model.grad_chunk and/or algorithm.method.num_particles; consider gradient checkpointing of the forward trajectory (noted in the plan).
- Sweep shows no objective: ensure the metric is 'relative l2' (already fixed in the sweep yaml) -- the AFDPS path does NOT log 'data_fitting_loss' (only the baselines do).
- Slow: use hutchinson_scheme=forward (halves Laplacian solves), raise grad_chunk to fill the GPU, and run the 9 grid cells in parallel across GPUs.

Key files & reference

- README.md -- overview + the same run commands.
- configs/{problem/navier-stokes-afdps.yaml, algorithm/afdps.yaml, sweep/navier-stokes/afdps.yaml}
- inverse_problems/{ns_adjoint.py, navier_stokes_afdps.py} -- gradient engine + operator.
- algo/{afdps.py, afdps_core/ensemble_denoiser_edm.py} -- the algorithm + sampler.
- verification/ -- checks.py, run_track_a.py, smoke_track_b.py.
- Plan: /Users/eric/.claude/plans/plan-out-the-entire-virtual-stearns.md Math recipe:

AFDPS_PDE_Inverse_Problem (3).pdf.